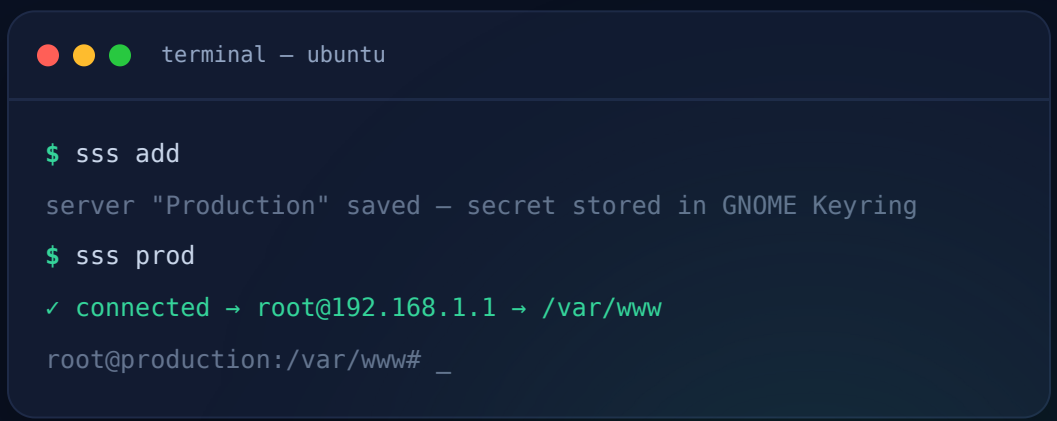
مدیر اتصال SSH حرفه‌ای برای اوبونتو

راهنمای کاربر

SSH Desk

از ثبت سرور تا شِل تعاملی واقعی — با یک دستور. رابط گرافیکی، خط فرمان و هستهٔ مشترک؛ رمزها فقط در GNOME Keyring و موتور SSH همان OpenSSH سیستم است.



- ترمینال PTY واقعی
- رمزها فقط در Keyring
- مدیر فایل SFTP
- تونل Local/Remote/SOCKS
- نصب آفلاین با میرورها

فهرست

در این راهنما

۰۱ معرفی SSH Desk

۰۲ نصب و راه‌اندازی

۰۳ شروع سریع: اولین سرور

۰۴ اتصال و مدیریت مسیرها

۰۵ مرجع کامل خط فرمان sss

۰۶ رابط گرافیکی

۰۷ ترمینال واقعی (PTY)

۰۸ مدیریت فایل با SFTP

۰۹ کلیدها، تونل‌ها و دستورات

۱۰ مدل امنیتی

۱۱ پشتیبان‌گیری و عیب‌یابی

فصل ۰۱

معرفی SSH Desk

SSH Desk یک محصول واقعی و قابل‌نصب برای مدیریت همهٔ سرورهای SSH شما روی اوبونتو است؛ نه یک نمونهٔ نمایشی. سه ماژول — رابط گرافیکی، خط فرمان و هستهٔ مرکزی — دقیقاً روی یک

دیتابیس و یک مخزن رمز مشترک کار می‌کنند: سروری را در GUI ثبت می‌کنید و در همان لحظه از هر ترمینالی با `sss prod` به آن وصل می‌شوید؛ حتی وقتی GUI اصلاً در حال اجرا نیست.

موتور اتصال، همان **OpenSSH سیستم** است؛ پروتکل SSH دوباره پیاده‌سازی نشده تا رفتار اتصال دقیقاً مثل خودِ اوبونتو باشد. ترمینال داخلی یک **PTY واقعی** است: رنگ‌ها، کلیدهای جهت‌دار، **Ctrl+C** و اجرای vim و nano همه به‌درستی کار می‌کنند. رمزهای عبور فقط در **GNOME Keyring** ذخیره می‌شوند و هرگز به دیتابیس، لاگ، آرگومان دستور یا متغیر محیطی راه نمی‌یابند.

GUI — رابط گرافیکی

PySide6 با تم تیره/روشن، ۷ زبان رابط کاربری (RTL کامل برای فارسی و عربی)، ترمینال چندزبانه و مدیریت فایل.

CLI — خط فرمان

دستور `sss` بدون نیاز به GUI؛ نام دستور قابل تغییر؛ تکمیل خودکار bash/zsh.

Core — هستهٔ مشترک

SQLite + SQLAlchemy با مهاجرت‌ها، سرویس Keyring، سازندهٔ دستور ssh و سیاست Host Key.

۳
ماژول با یک موتور

۹
صفحه در رابط گرافیکی

+۳۰
زیردستور خط فرمان

فصل ۰۲

نصب و راه‌اندازی

پیش‌نیازها

```
# Ubuntu 22.04 / 24.04+
sudo apt update
sudo apt install -y openssh-client gnome-keyring libsecret-1-0 \
     python3 python3-venv python3-pip
```

› گزینهٔ الف — نصب سطح کاربر (پیشنهادی، بدون sudo)

```
git clone https://github.com/hmplus28/SSH-Desk.git
cd SSH-Desk
bash scripts/install.sh
```

این اسکریپت پایتون و OpenSSH را بررسی می‌کند، محیط مجازی را در `~/.local/share/ssh-desk/venv` می‌سازد، بسته را با همهٔ وابستگی‌ها نصب می‌کند و در `~/.local/bin` میان‌برهای `ssh-desk` (رابط گرافیکی)، `ssh-desk-askpass`، `ssh-desk-cli` و `sss` را ایجاد می‌کند؛ سپس تکمیل خودکار bash و آیکن برنامه را هم نصب می‌کند.

› گزینهٔ ب — بستهٔ DEB سراسری

```
bash scripts/package_deb.sh
sudo apt install ./dist/ssh-desk_1.0.0_amd64.deb
```

بستهٔ DEB یک حباب مستقل PyInstaller را در `/opt/ssh-desk` نصب می‌کند و ورودی منو و آیکن سیستمی را ثبت می‌کند. ارتقا با نصب بستهٔ جدیدتر در جای خود انجام می‌شود و داده‌ها و رمزهای شما دست‌نخورده می‌مانند.

› اینترنت ضعیف یا قطع: میرورها و نصب آفلاین

اگر شبکهٔ شما ناپایدار است یا PyPI در آن به‌درستی در دسترس نیست، نگران نباشید: همهٔ فراخوانی‌های pip در اسکریپت‌های نصب به‌صورت خودکار زنجیره‌ای از **میرورها** را امتحان می‌کنند — اگر یک میرور در دسترس نباشد، بدون دخالت شما از آن عبور کرده و میرور بعدی را می‌آزمایند. ترتیب پیش‌فرض: **PyPI رسمی ← ArvanCloud (ایران) ← Tencent ← Aliyun ← TUNA**.

```
# force a specific working mirror
bash scripts/install.sh --mirror-name ir    # ArvanCloud (Iran)
bash scripts/install.sh --mirror-name tuna  # TUNA (China)

# 1) download everything once (on any working network)
bash scripts/download_wheels.sh --mirror-name ir
# 2) install later with NO network at all
bash scripts/install.sh --offline
```

در حالت pip ،`--offline` اجازهٔ هیچ اتصال شبکه‌ای ندارد و فقط از پوشهٔ `vendor/wheels` نصب می‌کند؛ بنابراین قطع شدن اینترنت وسط نصب، دیگر نمی‌تواند نصب را نیمه‌تمام رها کند. همین زنجیرهٔ میرور در `scripts/build.sh` هم استفاده می‌شود تا ساخت بستهٔ DEB نیز از قطعی شبکه آسیب نبیند.

› فایل‌های مهم کجایند؟

محل	مسیر
دیتابیس برنامه	`~/.local/share/ssh-desk/ssh-desk.db`
تنظیمات	`~/.config/ssh-desk/config.toml`
known_hosts برنامه	`~/.local/share/ssh-desk/known_hosts`
کلیدهای تولیدشده	`~/.local/share/ssh-desk/keys/`
لاگ‌ها	`~/.local/share/ssh-desk/logs/`
رمزها و صف‌های محرمانه	GNOME Keyring (Secret Service)

حذف برنامه: `bash scripts/uninstall.sh` پیش از پاک‌کردن داده‌ها و رمزها به‌صراحت از شما اجازه می‌گیرد؛ بدون تأیید، هیچ داده‌ای حذف نمی‌شود.

فصل ۰۳

شروع سریع: اولین سرور

گام ۱ — اجرای برنامه: با دستور `ssh-desk` رابط گرافیکی باز می‌شود؛ یا کاملاً در ترمینال بمانید و از `sss add` استفاده کنید.

گام ۲ — افزودن سرور: در صفحهٔ Servers دکمهٔ «Add server» را بزنید. نام، نام‌کوتاه (Alias — یکتا و همان چیزی که با آن وصل می‌شوید)، میزبان، پورت، نام‌کاربری و مسیر پیش‌فرض را وارد کنید. اگر روش احراز را «رمز عبور» انتخاب کنید، رمز همان لحظه در GNOME Keyring ذخیره می‌شود — نه در دیتابیس. پیش از ذخیره، دکمهٔ **Test** یک اتصال واقعی می‌آزماید تا مطمئن شوید همه‌چیز درست است.

```
# the same thing from the terminal (interactive wizard)
sss add
sss add --alias prod --host 192.168.1.1 --port 22 --user root \
        --auth password --path /var/www
```

گام ۳ — اتصال: از هر ترمینالی بنویسید `sss prod` تا شِل تعاملی روی مسیر پیش‌فرض باز شود. در GUI هم دوبار کلیک روی سرور کافی است تا یک زبانهٔ ترمینال واقعی باز شود.

اولین اتصال: اثرانگشت (Fingerprint) کلید میزبان نمایش داده می‌شود؛ آن را با مدیر سرور تطبیق دهید و یک‌بار بپذیرید. اگر بعدها کلید میزبان **تغییر** کرده باشد، اتصال بلافاصله رد می‌شود و راهنمای بررسی احتمال حملهٔ MITM نمایش داده می‌شود.

فصل ۰۴

اتصال و مدیریت مسیرها

هر سرور می‌تواند یک **مسیر پیش‌فرض** و هر تعداد **مسیر ذخیره‌شدهٔ نام‌دار** داشته باشد؛ مثلاً برای همان سرورِ تولید، مسیر `website ← /var/www/html` و `nginx ← /etc/nginx` را یک‌بار تعریف کنید و همیشه با یک کلمه به همان پوشه بپرید:

```
sss prod               # shell at the server's default path
sss prod nginx         # shell at the saved path named "nginx"
sss prod /var/www      # shell at any absolute path
sss prod --exec "uptime"           # run one command, no shell
sss prod --user ubuntu --port 2222 # per-run override (NOT saved)
sss prod --dry-run                 # print the exact ssh command
```

اگر مسیر درخواستی روی سرور وجود نداشته باشد، پیام خطا روشن و قابل‌فهم است — نه یک قطع‌شدگی مبهم. مسیرها با نقل‌قول امن (shlex) به سرور منتقل می‌شوند تا فاصله یا کاراکترهای خاص در نام پوشه مشکلی بسازند. گزینه‌های `--user` و `--port` فقط برای همان اجرا اعمال می‌شوند و هرگز در تنظیمات ذخیره نمی‌شوند.

› کدهای خروج CLI

کد	معنا
0	عملیات موفق
1	خطای زمان اجرا (اتصال، شبکه، Keyring و …)
2	ورودی نامعتبر یا استفادهٔ نادرست از دستور
130	قطع‌شدن با Ctrl+C

این کدها باعث می‌شوند `sss` در اسکریپت‌ها و اتوماسیون‌های شما هم رفتاری قابل‌اعتماد داشته باشد؛ مثلاً می‌توانید در CI یا cron بررسی کنید که اتصال موفق بوده است.

فصل ۰۵

مرجع کامل خط فرمان sss

نام دستور `sss` نمونه‌ای است؛ می‌توانید با `sss config cli-name cnc` آن را به هر نام دلخواه (مثلاً cnc یا sshgo) تغییر دهید — تغییر در سطح سیستم‌عامل اعمال می‌شود (میان‌بر جدید در `~/.local/bin`). هر توکن اول که زیردستور شناخته‌شده نباشد، به‌عنوان نام‌کوتاه سرور تفسیر می‌شود؛ یعنی `sss prod` همان `sss connect prod` است.

› سرورها

دستور	کار
`sss list`	جدول سرورها؛ با `--group --favorites --search` فیلتر می‌شود
`sss add`	افزودن تعاملی؛ همهٔ فیلدها را می‌توان با گزینه مستقیم داد
`sss info prod`	جزئیات کامل؛ `--show-secret` با چالش تایپ نام‌کوتاه
`sss edit prod`	ویرایش میدان‌ها؛ `--set-password` برای ثبت رمز جدید
`sss delete prod`	حذف با تأیید؛ رازِ Keyring هم پاک می‌شود
`sss search TERM`	جست‌وجو در نام، نام‌کوتاه، میزبان، کاربر و توضیح

› مانیتورینگ و پیکربندی

دستور	کار
`sss history`	تاریخچهٔ اتصال‌ها با فیلتر سرور/تعداد/وضعیت
`sss current`	نشست‌های فعال SSH و تونل‌های در حال اجرا
`sss config list`	همهٔ تنظیمات و مسیر فایل‌ها
`sss config set KEY VALUE`	تغییر تنظیمات
`sss config cli-name cnc`	تغییر نام دستور در سطح سیستم‌عامل
`sss config completions`	نصب تکمیل خودکار bash/zsh

دستور	کار
`sss keys generate`	تولید Ed25519/RSA/ECDSA در قالب OpenSSH
`sss keys import`	ثبت کلید موجود (با `--copy` در انبار برنامه با دسترسی 0600)
`sss keys export-pub`	استخراج کلید عمومی
`sss tunnel start|stop`	اجرای تونل به‌صورت دیمن جدا از ترمینال
`sss import --file ~/.ssh/config`	درون‌ریزی ssh config با پیش‌نمایش و کشف تکراری
`sss export FILE.toml`	پشتیبان‌گیری؛ رمزها فقط با `--include-secrets`
`sss restore FILE.toml`	بازگردانی پشتیبان
`sss install-cli`	بازسازی میان‌برهای خط فرمان

تکمیل خودکار: bash به‌طور خودکار نصب می‌شود (زیردستورها + نام‌کوتاه سرورها). برای zsh: `sss config completions --shell zsh --install` و سپس افزودن مسیر completions به `~/.zshrc` .

فصل ۰۶

رابط گرافیکی

پنجرهٔ اصلی یک نوار کناری جمع‌شونده با ۹ صفحه و یک نوار وضعیت (نشست‌ها و تونل‌های فعال) دارد. زبان برنامه در ۷ زبان — انگلیسی، فارسی، عربی، چینی، روسی، اسپانیایی و فرانسوی — ارائه می‌شود و برای فارسی و عربی چیدمان راست‌به‌چپ واقعی دارد؛ تم تیره/روشن هم از Settings تغییر می‌کند.

صفحه	کارکرد
Dashboard	نمای کلی سرورها، نشست‌ها و تونل‌های فعال
Servers	جست‌وجو، فیلتر گروه/برچسب/علاقه‌مندی، Duplicate، تست سریع
Add/Edit Server	چهار بخش: عمومی، احراز هویت، اتصال و مسیرها، پیشرفته (Jump Host و گزینه‌های ssh)
Server Details	جزئیات؛ رمز به‌صورت ماسک‌شده و فقط با چالش تایپ نام‌کوتاه نمایان می‌شود
Terminal	زبان‌های ترمینال واقعی PTY با چند زبانه
Files	مدیر فایل دوپنجره‌ای SFTP با انتقال بین دو سرور
SSH Keys / Port Forwarding / Commands / History	مدیریت کلیدها، تونل‌ها، دستورات ذخیره‌شده و تاریخچهٔ اتصال‌ها
Settings	نام دستور CLI، تم، زبان، پیش‌فرض‌ها، وضعیت Keyring، پشتیبان‌گیری و درون‌ریزی ssh config

میان‌برهای صفحه‌کلید

`Ctrl+1..9` جابه‌جایی بین صفحات · `Ctrl+N` سرور جدید · `Ctrl+K` نشست ترمینال جدید · `Ctrl+Q` خروج.

> **هم‌زمانی کامل GUI و CLI:** هر دو رابط از یک دیتابیس، یک Keyring و یک تنظیمات استفاده می‌کنند؛ سروری که در GUI می‌سازید بلافاصله برای `sss` در همهٔ ترمینال‌ها شناخته می‌شود و برعکس.

فصل ۰۷

ترمینال واقعی (PTY)

هر زبانهٔ ترمینال یک نشست PTY واقعی روی OpenSSH است — نه یک جعبهٔ متن. هر چیزی که در یک ترمینال واقعی ممکن است، اینجا هم ممکن است:

- رنگ‌ها، کلیدهای جهت‌دار، تکمیل با Tab، و اجرای **nano**، **vim** و **htop** به‌درستی کار می‌کنند.
- **Ctrl+C** و **Ctrl+D** فرایندها را مثل ترمینال سیستم قطع و می‌بندند.
- **کپی:** با ماوس متن را انتخاب کنید (کپی خودکار) یا از نوار ابزار استفاده کنید؛ **پیست:** کلیک میانی، نوار ابزار یا `Ctrl+V` داخل ترمینال.

- **تغییر اندازه:** کشیدن پنجره، برنامهٔ سمت سرور را بی‌درنگ مطلع می‌کند (SIGWINCH) — ویرایشگرها و htop درست بازترسیم می‌شوند.
- عنوان هر زبانه نام سرور و عنوان پنجرهٔ سمت سرور را نشان می‌دهد؛ برای اتصال مجدد کافی است نشست جدیدی از همان سرور باز کنید.

حریم خصوصی: محتوای ترمینال هرگز ثبت (لاگ) نمی‌شود؛ فقط وضعیت اتصال و مدت آن در تاریخچه می‌آید.

فصل ۰۸

مدیریت فایل با SFTP

از صفحهٔ Files سرور را انتخاب و Connect بزنید: پنجرهٔ چپ سیستم محلی و پنجرهٔ راست سرور است. ناوبری با دوبار کلیک روی پوشه‌ها، دکمهٔ «بالا» یا ویرایش مستقیم مسیر انجام می‌شود. همهٔ عملیات روزمره پشتیبانی می‌شود:

- **بارگذاری و دانلود** با کشیدن و رها کردن یا دکمه‌ها.
- **پوشهٔ جدید، تغییر نام، حذف** (با تأیید و به‌صورت بازگشتی).
- **تغییر مجوزها** به‌صورت هشت‌هشتی (مثلاً 0644) — بدون حفظ‌کردن اعداد در ذهن.
- **ویرایش فایل متنی:** فایل به‌صورت محلی باز و ویرایش می‌شود و پس از ذخیره، آپلود مجدد پیشنهاد می‌شود.
- **«ارسال به سرور...»:** یک فایل را مستقیم بین دو سرورِ ثبت‌شده جابه‌جا کنید — بدون دانلود به سیستم میانی.

اگر SFTP کار نمی‌کند ولی ترمینال کار می‌کند: ممکن است سرور زیرسیستم SFTP را غیرفعال کرده باشد (`Subsystem sftp` در sshd_config) یا کلید میزبان هنوز پذیرفته نشده باشد — یک‌بار با ترمینال وصل شوید یا Test بزنید.

فصل ۰۹

کلیدها، تونل‌ها و دستورات ذخیره‌شده

کلیدهای SSH

کلید Ed25519 (پیشنهادی)، RSA یا ECDSA را در قالب استاندارد OpenSSH تولید کنید؛ گذرواژهٔ اختیاری کلید هم در Keyring ذخیره می‌شود. کلیدهای موجود را «ثبت» کنید (بدون کپی‌کردن فایل) یا با انتخاب خودِ شما با دسترسی 0600 در انبار برنامه کپی شوند. ستون اثرانگشت برای راستی‌آزمایی و ستون

«سرورهای متصل» برای دیدن مصرف‌کنندگان هر کلید هست؛ «نصب روی سرور» دستور `ssh-copy-id` را می‌سازد و در یک نشست ترمینال اجرا می‌کند.

```
sss keys generate --name deploy --type ed25519
sss keys import --path ~/.ssh/id_ed25519 --name personal
sss keys export-pub deploy
```

› تونل‌های پورت

سه نوع تونل: **Local** (مثلاً `127.0.0.1:5432 → db-internal:5432` برای رسیدن به دیتابیس داخلی)، **Remote** (معکوس، بازکردن پورت سرویس محلی روی سرور) و **Dynamic** (پروکسی SOCKS5 مثل `127.0.0.1:1080`). تونل‌ها در یک دیمن مستقل اجرا می‌شوند — با بستن GUI یا CLI نمی‌میرند و وضعیتشان در `sss current` دیده می‌شود.

```
sss tunnel db       # start the tunnel named "db"
sss tunnel stop db
sss current         # live sessions + running tunnels
```

› دستورات ذخیره‌شده

دستورهای پرتکرار را برای هر سرور یا سراسری ذخیره کنید (نام، دستور، دسته). دستورهای حساس با پرچم «نیاز به تأیید» قبل از اجرا می‌پرسند. اجرا از صفحهٔ Commands در GUI یا با `sss prod --exec "<command>"` در ترمینال ممکن است.

فصل ۱۰

مدل امنیتی

SSH Desk برای نگه‌داری اعتبارنامه‌های سرورهای تولیدی طراحی شده و از اول تا آخر بر یک اصل بنا شده است: **راز فقط در Secret Service می‌ماند**. مهم‌ترین کنترل‌ها:

۱ — رمزها فقط در GNOME Keyring

دیتابیس فقط یک ارجاع بدون‌راز (JSON) نگه می‌دارد؛ خودِ رمز هرگز به SQLite نمی‌رسد (با تست خودکار راستی‌آزمایی شده است).

۲ — askpass امن، نه sshpass

OpenSSH با `SSH_ASKPASS_REQUIRE=force` و helper امن اجرا می‌شود و راز را «با ارجاع» از Keyring می‌خواند؛ رمز در argv، محیط، لاگ یا فایل موقت نمی‌آید.

۳ — Host Key همیشه سخت‌گیرانه

`StrictHostKeyChecking=yes` در سازندهٔ دستور قطعی است و خاموش‌کردنش ممکن نیست؛ میزبان ناشناس ← پنجرهٔ اثرانگشت؛ تغییر کلید ← هشدار MITM و راهنمای `ssh-keygen -R` .

۴ — مقاوم به تزریق

همهٔ فراخوانی‌های subprocess با آرایهٔ آرگومان‌ها و بدون shell=True انجام می‌شوند؛ مسیرهای راه‌دور با shlex.quote نقل‌قول می‌شوند و همهٔ ورودی‌ها با Pydantic اعتبارسنجی می‌شوند.

سایر کنترل‌ها

- مجوز فایل‌ها: دیتابیس و پیکربندی 0600، پوشه‌های داده 0700، کلیدهای خصوصی 0600.
- لاگر همهٔ الگوهای `secret=` ، `password=` و رازهای ثبت‌شدهٔ زمان اجرا را حذف می‌کند.
- نمایش/کپی رمز نیازمند چالش تایپ نام‌کوتاه است و کلیپ‌بورد پس از ۳۰ ثانیه خودکار پاک می‌شود.
- عملیات حساس — حذف سرور، حذف بازگشتی فایل، پاک‌سازی کامل داده (تایپ PURGE) — همیشه تأیید می‌خواهند.
- هیچ دیمنی با دسترسی root اجرا نمی‌شود؛ GUI و CLI هرگز sudo نمی‌خواهند.
- تولید کلید کاملاً درون‌فرایندی است؛ گذرواژهٔ کلید هم به argv یا فایل موقت نمی‌رسد.
- خروجی/پشتیبان به‌طور پیش‌فرض بی‌رمز است؛ گنجاندن رمزها opt-in با هشدار صریح است.

فصل ۱۱

پشتیبان‌گیری، انتقال و عیب‌یابی

پشتیبان و بازگردانی

```
sss backup backup.toml                  # no secrets by default
sss restore backup.toml
sss export full.toml --include-secrets  # opt-in + warning
sss import --file ~/.ssh/config         # preview, then: --apply
```

درون‌ریزی از `~/.ssh/config` با پیش‌نمایش و کشف سرورهای تکراری انجام می‌شود؛ خروجی TOML برای انتقال تنظیمات به ماشین دیگر است و تا خودتان نخواهید، رمزی در آن نیست.

‹ خطاهای رایج و راه‌حل

علامت	راه‌حل
`sss: command not found`	`~/.local/bin` را به PATH اضافه کنید یا `sss install-cli` بزنید
«Secret Service is not available»	`sudo apt install gnome-keyring libsecret-1-0` و یک‌بار ورود مجدد
Keyring قفل است	یک‌بار بازگشایی کنید؛ برنامه هم در صورت امکان تعاملی می‌پرسد
Host key changed	فقط پس از راستی‌آزمایی: `ssh-keygen -R "[host]:port"` و سپس پذیرش دوباره
Permission denied	رمز: `sss edit prod --set-password` · کلید: `sss keys list`
GUI باز نمی‌شود	`ssh-desk 2>&1 | tail -20`؛ کتابخانه‌های Qt را نصب کنید؛ Wayland: `QT_QPA_PLATFORM=xcb`
پورت تونل اشغال است	`ss -tlnp | grep <port>` و سپس `sss tunnel stop`

بازنشانی کامل: `sss uninstall-data` — برای پاک‌سازی داده‌ها و رازهای Keyring باید PURGE را تایپ کنید.

اتصال امن، همیشه در دسترس.

SSH Desk متن‌باز است تحت مجوز MIT. اگر این راهنما به کارتان آمد، برای گزارش مسئله یا پیشنهاد ویژگی جدید به مخزن GitHub سر بزنید.

github.com/hmplus28/SSH-Desk